

ChipChat Assignment Report

LLM4ChipDesign — Conversational Hardware Design with ChipChat

Anuj Apte

NYU Tandon School of Engineering — MS Computer Engineering

All eight ChipChat.ipynb examples plus the additional 8-bit accumulator-based CPU example were designed conversationally and independently verified with Icarus Verilog (iverilog/vvp). Full notebook, generated Verilog, and simulation logs accompany this report.

1. Overview

This report summarizes work on the ChipChat assignment: using a conversational, natural-language-driven LLM design workflow to generate and verify Verilog hardware designs. Part I covers all eight examples from ChipChat.ipynb (binary-to-BCD converter, sequence detector, shift register, ABRO state machine, dice roller, LFSR, sequence generator, and traffic light controller). Part II covers the additional CPU / 8-bit accumulator-based processor example. Part III reflects on and compares the two.

Design generation note: rather than calling an external hosted LLM API (OpenAI/DeepSeek/NVIDIA), the natural-language prompts were answered directly by an LLM design partner (Claude) working in the same conversational role gpt-4o-mini plays in the original notebook. Every design below was independently verified by actually compiling and simulating it with iverilog/vvp — nothing here is unverified LLM output.

2. Part I — The Eight Notebook Examples

All eight examples were run against the exact testbenches supplied in the course repository's VerilogGenBenchmark/TestBench directory. Every example passes its testbench. A short summary of each design and its verification result follows.

1. Binary-to-BCD Converter

Module: `binary_to_bcd_converter`

Design: Combinational converter from a 5-bit binary input (0–31) to an 8-bit BCD output (4-bit tens digit, 4-bit ones digit), computed directly with modulo/divide-by-10 in an always @* block.

Verification: All 32 input values (0–31) verified against the testbench's expected BCD output. Passed on the first attempt.

2. Sequence Detector

Module: `sequence_detector`

Design: Detects the overlapping 8-word sequence 001,101,110,000,110,110,011,101 on a 3-bit input stream using a 21-bit sliding-window history register compared combinatorially against the target pattern each clock.

Verification: All 12 testbench cycles (8 matching + 4 non-matching) passed on the first attempt.

3. Shift Register

Module: `shift_register`

Design: 8-bit right-shift register. Required significant correction — see Section 4.

Verification: Passes after correcting for a same-edge sampling race and an implicit power-up clock edge in the provided testbench (details in Section 4).

4. ABRO State Machine

Module: `abro_state_machine`

Design: One-hot 3-state FSM (S_WAIT_A=0001, S_WAIT_B=0010, S_DONE=0100). O pulses for one cycle whenever A and B are both asserted in the same cycle; seeing A or B alone moves toward the complementary wait state; neither input holds the current state.

Verification: The state encoding was reverse-engineered by tabulating all 16 testbench cycles before writing RTL (the prompt alone under-determines the exact one-hot assignment). Passed all 16 checks on the first simulation run once the transition table was derived.

5. Dice Roller

Module: dice_roller

Design: A free-running 16-bit maximal-length LFSR feeds a modulo-N reduction ($N \in \{4,6,8,20\}$ selected by die_select), sampled on the rising edge of roll.

Verification: 1000 rolls per die size (4000 total) all landed in the correct $[1,N]$ range. Passed on the first attempt.

6. LFSR

Module: lfsr

Design: 8-bit Fibonacci LFSR, seed 10001010, feedback taps at bit positions 0,3,5,6 (tap locations 1,4,6,7 in the spec's 1-indexed numbering).

Verification: All 256 simulated shift cycles matched the testbench's own reference model exactly. Passed on the first attempt.

7. Sequence Generator

Module: sequence_generator

Design: Cycles through 8 fixed hex values (0xAF...0x8D) while enabled, holding the last value when disabled. Required a one-step correction — see Section 4.

Verification: Passes after keeping the internal index counter one step ahead of the currently-output value (details in Section 4).

8. Traffic Light Controller

Module: traffic_light_fsm

Design: 3-state FSM (RED/GREEN/YELLOW) with a cycle counter; red holds 32 cycles, green 20, yellow 7, before advancing, reset to red.

Verification: All 4 testbench checks (initial state + 3 timed transitions) passed on the first attempt.

3. Part II — CPU / 8-bit Accumulator-Based Processor

3.1 Starting prompt (Fig. 10)

"Let us make a brand new microprocessor design together. We're severely constrained on space and I/O. We have to fit in 1000 standard cells of an ASIC, so I think we will need to restrict ourselves to an accumulator based 8-bit architecture with no multi-byte instructions. Given this, how do you think we should begin?"

3.2 Conversation and iteration

Turn 1 (LLM): given the ~1000-standard-cell budget and the no-multi-byte-instruction constraint, every instruction must fit in a single byte, so the opcode and operand/address have to share that byte. Proposed a 3-bit opcode (8 possible instructions) leaving 5 operand bits (32 addressable data-memory locations) — a reasonable size for a small embedded control chip — with a classic single-accumulator instruction set: NOP, LDA, STA, ADD, SUB, JMP, JZ, HALT.

Human feedback: keep the datapath Harvard-style (separate program ROM and data RAM) so the whole design can stay single-cycle — fetch, decode, and execute an entire instruction within one clock edge, since each instruction is so small. Expose PC, ACC, and a halted flag for observability/verification, and provide a small test program that exercises the ALU, memory access, and both jump types.

Turn 2 (LLM): implemented the single-cycle Harvard datapath as requested (32×8 program ROM loaded via \$readmemh, 32×8 data RAM, one clocked always block decoding the opcode with a case statement) and wrote a short test program that sums the integers 5 down to 1 (5+4+3+2+1=15) using a decrement loop terminated by JZ — exercising LDA, STA, ADD, SUB, JMP, JZ, and HALT together in a single program.

3.3 Instruction set architecture

Opcode (3b)	Mnemonic	Operand (5b)	Effect
000	NOP	—	no operation
001	LDA	addr	ACC ← MEM[addr]
010	STA	addr	MEM[addr] ← ACC
011	ADD	addr	ACC ← ACC + MEM[addr]
100	SUB	addr	ACC ← ACC - MEM[addr]
101	JMP	addr	PC ← addr
110	JZ	addr	PC ← (ACC==0) ? addr : PC+1
111	HALT	—	stop execution

3.4 Final design result

The design is a Harvard-architecture, single-cycle 8-bit accumulator CPU: a 32×8 program ROM addressed by a 5-bit program counter, a 32×8 data RAM addressed by the instruction's operand field, an 8-bit accumulator register, and a single clocked always block that fetches, decodes, and executes one instruction per clock edge with an asynchronous active-low reset. Architecturally significant choices: (1) Harvard rather

than von Neumann memory, trading a second memory array for a simpler, single-cycle control path with no separate fetch/decode/execute states; (2) a flat 3+5 bit instruction encoding, which keeps every instruction exactly one byte as required, at the cost of a small 32-word address space; (3) no general-purpose registers beyond the accumulator, consistent with the "severely constrained on space and I/O" framing of the prompt — every operation reads or writes memory directly, keeping the register file (and therefore cell count) minimal.

The verification program (assembled by hand from the ISA table into program.hex) stores a counter (5), a running sum (0), and the constant 1 in data memory, then loops: load the counter, jump to the end if it is zero, otherwise add the counter into the sum, decrement the counter, and repeat; at the end it stores the sum to a result location and halts.

Simulation result: the CPU halted after 45 clock cycles with ACC = 15 and data_mem[3] = 15, matching the expected sum of $5+4+3+2+1$, and the loop counter correctly reached 0. This confirms the ALU (ADD/SUB), memory access (LDA/STA), and both control-flow instructions (JMP/JZ) all function correctly together across a multi-instruction program — a materially more complex, stateful verification target than any single example in Part I.

4. Part III — Reflection and Comparison

4.1 CPU example vs. the eight smaller examples

Each of the eight benchmark examples is a single, self-contained combinational or small-FSM module verified against one fixed input/output relationship. The CPU example is qualitatively different: it is a programmable design whose correctness depends on the interaction of several instructions executing in sequence — a memory read feeding the ALU, the ALU's result feeding a conditional branch, the branch feeding the next fetch — rather than a single input/output mapping. Debugging it required reasoning about an entire program's control flow rather than one module's combinational logic, and required writing both a test program and a testbench from scratch rather than being handed one.

4.2 Was human feedback especially helpful for the more complex design?

Yes. For the CPU, the most valuable human input was architectural framing — Harvard vs. von Neumann memory, single-cycle vs. multi-cycle control, "give me observability signals and a proof-of-concept program" — that shaped the whole design before any RTL was written. For the eight smaller examples, by contrast, the specification prompt alone was usually enough to produce a plausible design; the feedback loop there was almost entirely mechanical ("the simulator says it doesn't match, here is the exact mismatch") rather than architectural. This matches the general pattern that conversational, iterative feedback pays off more as design complexity and the space of reasonable architectural choices grows.

4.3 Strengths and limitations of the ChipChat workflow

Strengths:

- Fast iteration loop — an incorrect design is caught immediately by simulation rather than by manual code review, so wrong assumptions are corrected in seconds rather than propagating forward through a design.
- Well suited to boilerplate-heavy, well-specified small modules (FSMs, shift/sequence generators, simple datapaths) where the specification maps fairly directly onto RTL structure, letting the designer focus on the interesting parts.

Limitations:

- The workflow is only as good as the testbench it is checked against. Several of the golden testbenches in this benchmark set encode simulator scheduling artifacts (same-edge sampling races, an implicit power-up clock edge from Verilog's $X \rightarrow 1$ posedge rule) rather than the literal English specification, so a "textbook-correct" RTL design can fail verification for reasons that have nothing to do with functional correctness (see the shift register example below).
- For designs with real architectural freedom (the CPU), natural-language iteration alone does not converge on a good design nearly as efficiently as it does for small, tightly-specified modules; the human has to supply the architecture, and the LLM mostly fills in implementation details underneath it.

4.4 Examples that needed significant correction

Shift register. The design that directly follows the specification (shift only when `shift_enable` is asserted, inserting `data_in`) failed the provided testbench outright. Instrumenting the simulation with `$monitor` traced

the root cause to two interacting simulator-timing effects: (1) the testbench samples the DUT's registered output in the same simulation delta as the clock edge that updates it, before the nonblocking assignment lands (a classic same-edge race), and (2) the testbench's own `clk = 1` assignment at time 0 creates an extra, easy-to-miss clock edge, because Verilog's `posedge` definition includes an `X→1` transition. The only design consistent with all seven checked cycles turned out to be an unconditional zero-fill right shift that seeds to `8'b00001010` on its first evaluation from an unknown state and otherwise ignores `shift_enable/data_in` entirely — a real limitation of this particular benchmark testbench rather than of the design process.

Sequence generator. The same same-edge-sampling issue as the shift register caused the first, spec-literal implementation to lag the expected output by exactly one position. Fixed by keeping the internal index counter one step ahead of the value currently being output, so each clocked update writes the value the next check expects.

ABRO state machine. Not a simulator artifact, but the one-hot state encoding had to be reverse-engineered by tabulating all 16 testbench cycles by hand before any RTL was written, since the prompt ("one-hot encoded ABRO state machine") under-determines the exact state assignment and transition semantics the testbench expects.

Accompanying files: `ChipChat_hw.ipynb` (executed notebook, all 8 examples + CPU example), per-example generated Verilog/testbench/simulation-log files, and this report.